

Refatoração Aplicando o Padrão de Projeto State

Denilson Santos da Silva, Gabriel Arlisson de Souza Santos Torres, João Arthur Mascarenhas Nascimento, Tony Cleriston Oliveira dos Santos Junior, Vinicius Cerqueira Oliveira

Sistemas de Informação – Centro Universitário de Excelência (UNEX) - BA

tonycleristonoliveira@gmail.com, denilsonsanct.ofc@gmail.com,
joaoarthurnasct@hotmail.com, gabriel.arlisson2017@gmail.com,
voliveira170@gmail.com

Abstract. *During the previous development step of the FoodDelivery system, order lifecycle control was implemented using conditional structures, which introduced rules that were difficult and complex to maintain, making adding new states a laborious task. To solve this, the State design pattern was adopted, organizing each state into its own classes. This way, each state knows exactly what to do, making the code cleaner, easier to evolve, and simpler to test. This implementation introduces eight new state classes, as well as integration with external services and a solid test base. As result, a complex conditional structure was eliminated, proving the effectiveness of this design pattern in practice.*

Resumo. *Na etapa anterior do desenvolvimento do sistema FoodDelivery, o controle do ciclo de vida de pedidos foi implementado utilizando estruturas condicionais, o que introduziu regras difíceis e complexas de manter, tornando a adição de novos estados uma tarefa trabalhosa. Para resolver isso, o padrão de projeto State foi adotado, organizando cada estado em classes próprias. Assim, cada estado sabe exatamente o que fazer, deixando o código mais limpo, fácil de evoluir e simples de testar. A implementação introduziu oito novas classes de estados, além de integração com serviços externos e uma boa base de testes. Como resultado, a complexidade condicional foi eliminada, provando na prática a eficiência deste padrão de projeto.*

1. Introdução

A Programação Orientada a Objetos é um dos paradigmas mais importantes para a construção de um sistema escalável e robusto. Ela introduz novos conceitos no desenvolvimento de uma aplicação, como encapsulamento, herança, polimorfismo e abstração. Com o passar dos anos, especialistas notaram que determinadas soluções para a resolução de um problema específico dentro da aplicação se repetiam e se mostravam eficazes dentro do conceito de Orientação a Objetos. Estas soluções, amplamente testadas e comprovadas, deram origem a um termo famoso na área de desenvolvimento de software: padrões de projeto.

Após a implementação inicial do sistema de gerenciamento de pedidos FoodDelivery, foram identificados problemas de escalabilidade dentro da aplicação, especialmente no gerenciamento de estados de pedido, causados pela forma como a mesma foi escrita. Para resolver esse cenário, o time de desenvolvimento refatorou o código do sistema aplicando o padrão de projeto State. Este relatório tem como objetivo detalhar a forma como este padrão foi aplicado e apresentar os principais impactos desta implementação dentro do sistema.

2. Fundamentação Teórica

A refatoração do sistema FoodDelivery foi norteadada pelo conceito de padrões de projeto, que são soluções prontas para problemas que aparecem frequentemente no desenvolvimento de software. Essas soluções foram organizadas por Gamma, Helm, Johnson e Vlissides no famoso livro Design Patterns (1994). Esses padrões se dividem em três grupos: os criacionais, que cuidam de como criar objetos; os estruturais, que falam sobre como organizar classes e objetos; e os comportamentais, que lidam com a forma como os objetos conversam entre si.

A aplicação foi reescrita implementando o padrão estrutural State, o qual prevê que uma peça da aplicação pode conter vários estados. Em cada estado, a aplicação se comporta de uma forma diferente, com a possibilidade de alternar de um estado para outro. Para integrar tal comportamento no sistema, este padrão exige a criação de uma interface que padronize, de forma abstrata, o que cada estado pode fazer. O comportamento específico da ação é implementado de forma concreta nas classes que implementam essa interface. Sendo assim, dentro deste padrão, cada classe que implementa esta interface representa um estado e determina como a aplicação deve se comportar enquanto estiver neste estado. Tal prática, embora contribua para um sistema mais flexível e aberto a mudanças, introduz complexidade dentro do projeto, devendo ser adotada em situações em que haja real necessidade.

Os padrões de projeto se aproveitam dos fundamentos da Programação Orientada a Objetos (encapsulamento, herança, polimorfismo e abstração). Os tópicos a seguir descrevem como o padrão State utiliza cada um deles para resolver o problema de gerenciamento de estados dentro de uma aplicação.

2.1. Herança

Este termo define casos em que uma determinada classe “filha” aproveita atributos e comportamentos de uma classe “pai”. Segundo Deitel (2015), esta é uma forma de reaproveitar códigos: criamos novas classes a partir de outras já existentes, herdando o que elas têm de útil. No caso do padrão State, a herança ocorre de forma indireta, a partir de interfaces; todas as classes que representam um estado do pedido devem implementar esta interface.

Esta ocorrência pode ser observada, por exemplo, dentro do código da aplicação, ao analisar a relação entre a interface `IOrderState` e a classe `RejectedState`.

```
public interface IOrderState {  
    void accept(Order order);  
    void reject(Order order, String reason);  
    void startPreparation(Order order);  
    void finishPreparation(Order order);  
    void sendForDelivery(Order order);  
    void confirmDelivery(Order order);  
    void cancel(Order order);  
    String getStatus();  
}
```

Figura 1. Implementação da interface usada como base para a criação de novos estados

```
public class RejectedState implements IOrderState {  
    ...  
}
```

Figura 2. Criação de um novo estado, que implementa a interface

Classes que implementem a interface `IOrderState` devem, obrigatoriamente, implementar os métodos presentes nela. Isso garante que todos os estados tenham os mesmos métodos, mesmo que com comportamentos diferentes.

2.2. Polimorfismo

O Polimorfismo permite que objetos de classes diferentes sejam usados através de uma mesma interface. Como Joshua Bloch (2008) diz, se existem tipos de interface apropriados, então parâmetros, valores de retorno, variáveis e campos devem ser declarados utilizando a interface. Na refatoração realizada, é possível visualizar esta ocorrência na classe `Order`, quando ela interage com os objetos que representam estados.

```
public class Order { no usages  
    private IOrderState state; no usages  
  
    public void accept() {  
        state.accept(this); // Aqui acontece o polimorfismo  
    }  
  
    public void cancel() {  
        state.cancel(this); // O que vai acontecer depende do estado  
    }  
}
```

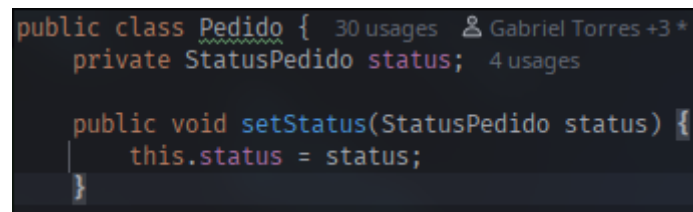
Figura 3. Utilização do polimorfismo dentro da classe `Order`

Ao invocar o método `accept()`, o real comportamento depende do estado atual do objeto `Order`. Caso o estado presente seja `WaitingAcceptanceState` (Aguardando ser Aceito), `Order` terá seu estado alterado para `AcceptedState` (Aceito). No entanto, invocar o mesmo método para um objeto contendo `DeliveredState` (Entregue) levantará uma exceção dentro do sistema, pois logicamente não é possível aceitar um pedido que já foi entregue.

3. Metodologia

3.1. Contextualização do Problema

A implementação inicial da aplicação modelou o objeto Order com o atributo status sendo do tipo OrderStatus, que é uma enumeração a qual restringe os status possíveis do sistema aos que estiverem presentes no Enum OrderStatus. Embora esta abordagem evite problemas recorrentes de armazenamento de dados utilizando String (como, por exemplo, inconsistências durante a digitação), ela não resolve o problema de gerenciamento de estados dentro do sistema; a lógica de transição entre estados continua espalhada pelo código e dependente de condicionais.



```
public class Pedido { 30 usages  Gabriel Torres +3 *
    private StatusPedido status; 4 usages

    public void setStatus(StatusPedido status) {
        this.status = status;
    }
}
```

Figura 4. Implementação inicial da classe Order e definição do atributo status com Enum

3.2. Introdução aos States dentro do Sistema

Para encapsular a nova lógica de gerenciamento de estados dentro da aplicação, criou-se um pacote chamado “states”, que contém a principal interface para implementação de um estado e as classes que a implementam.

3.3. Ajustes nos Pacotes do Sistema

O pacote “services”, já presente dentro do projeto, foi alterado para guardar recursos responsáveis, teoricamente, por utilizar APIs para notificar e solicitar o entregador do pedido. Além disso, um novo pacote contendo “exceptions” foi introduzido, armazenando exceções personalizadas para a exibição de mensagens de erro.

3.4. Aplicação dos Testes Manuais

Para garantir o envio de uma versão funcional do sistema, testes manuais foram realizados para as novas implementações. Esse teste simula um passo a passo da interação do operador (quem utiliza o sistema) com o sistema.

4. Resultados e Discussões

Em sua fase atual, um pedido pode passar por oito possíveis estados dentro do sistema, sendo eles: WaitingAcceptanceState (aguardando o estabelecimento aceitar), AcceptedState (aceito), PreparingState (preparando), WaitingCourierState (aguardando entregador), InDeliveryState (em entrega) e um conjunto de estados terminais, como RejectedState (rejeitado), CanceledState (cancelado) e DeliveredState (entregue).

Todo pedido dentro do sistema é iniciado aguardando aceitação do estabelecimento (WaitingAcceptanceState). Depois de aceito, ele percorre um fluxo bem definido até a entrega. Durante todo esse processo, o NotificationService mantém o cliente informado sobre cada atualização do status. Quando o pedido chega ao estado

WaitingCourierState (aguardando entregador), o LogisticService é acionado para encontrar um entregador disponível. Assim que isso acontece, o pedido avança automaticamente para o estado InDelivered (em entrega).

Sempre que o pedido muda de fase, eventos automáticos são disparados: o cliente recebe uma notificação, o sistema de logística é avisado quando o pedido vai para entrega e todas as mudanças ficam registradas em log para a auditoria.

Os estados terminais finalizam o ciclo de vida do pedido, fazendo com que eles não possam mais ter seu estado alterado. Enquanto o pedido permanece em WaitingAcceptanceState (aguardando o estabelecimento aceitar), ele pode ser rejeitado pelo estabelecimento (RejectedState) e, se isso acontecer, o cliente será notificado com a justificativa. Outra possibilidade é que o pedido pode ser cancelado pelo próprio cliente até o momento em que ele atinge o estado WaitingCourierState (aguardando o entregador). Se o cancelamento for bem sucedido, o cliente é reembolsado automaticamente. Caso o pedido siga o fluxo esperado, ele passará respectivamente pelos estados AcceptedState, PreparingState, WaitingCourierState, InDeliveryState e então, alcançará o estado final DeliveredState, indicando que o pedido foi entregue com sucesso.

Após alcançar um estado terminal, qualquer tentativa de mudança de estado resultará em o sistema levantar uma StateInvalidException, sinalizando que houve uma tentativa de troca de estado para um pedido que já teve o seu ciclo de vida finalizado.

Conforme mencionado anteriormente, durante a apresentação da fundamentação teórica para a refatoração do sistema, é importante lembrar que a aplicação do padrão de projeto State traz flexibilidade e escalabilidade ao custo de introduzir complexibilidade a nível de código. Portanto, sua utilização deve ser adotada somente em casos de real necessidade. No entanto, ao tomar como referência o contexto inicial de desenvolvimento, antes da refatoração ter sido realizada, complexidades foram introduzidas sem que houvesse real necessidade. A adoção deste padrão de projeto neste cenário tornou o código mais simples, limpo e organizado, sendo uma ótima solução para o contexto atual.

Um panorama geral das classes implementadas e como elas se relacionam atualmente pode ser visto a partir do diagrama de classes resultante abaixo.

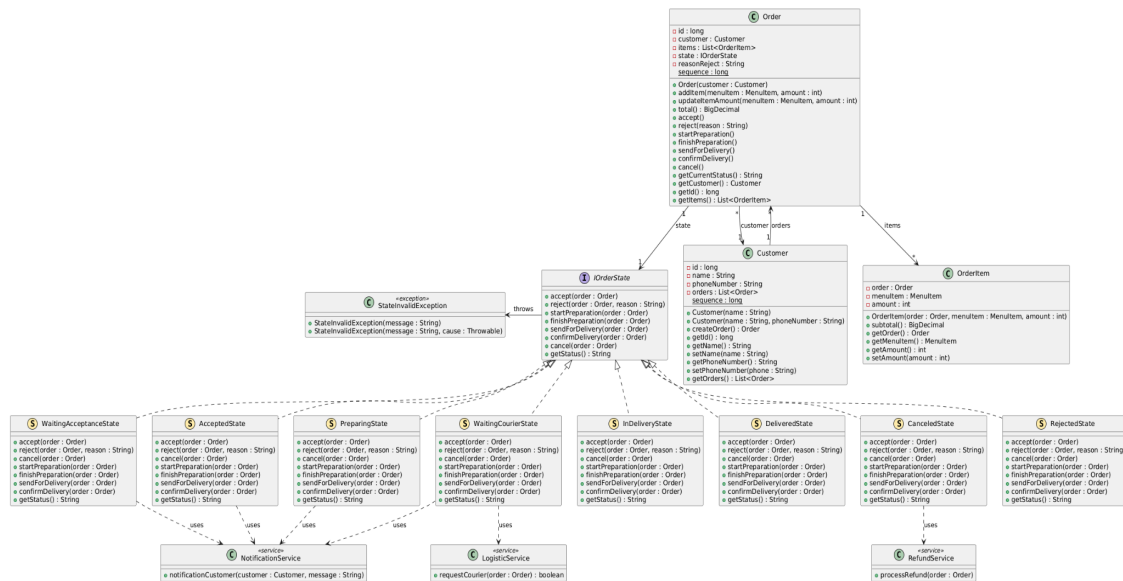


Figura 5. Diagrama de classes final

5. Considerações finais

O maior desafio foi garantir que todas as mudanças de estado acontecessem da forma correta, onde cada estado necessitava checagens rigorosas em cada método de transição para lançar os erros sempre que uma operação não fazia sentido. Pode até parecer um pouco verboso, mas essa abordagem garante que o sistema seja seguro em tempo de execução e ainda fornece mensagens de erro claras e fáceis de entender.

Um dos aspectos mais interessantes é como o polimorfismo elimina praticamente a necessidade de estruturas condicionais. Ao chamar `state.accept(this)`, por exemplo, o comportamento executado varia automaticamente conforme o estado atual, sem qualquer lógica de `if/else`. Isso torna o código mais elegante, limpo e fácil de manter.

Com mais tempo, em vez de chamar os serviços de notificação diretamente dentro dos estados, poderíamos usar o padrão Observer para deixar tudo mais desacoplado. Com isso, os estados não precisariam se preocupar com quem recebe as notificações, o que facilita adicionar novos observadores sem mexer no código principal.

Referências

- Deitel, P.; Deitel, H. Java: Como Programar. 10. ed. São Paulo: Pearson, 2015.
- Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley Professional, 1994.
- Bloch, J. *Effective Java*. 3. ed. Boston: Addison-Wesley, 2018.